



DSTI

Data ScienceTech Institute

School of Engineering

Paris, France

Project Report — Machine Learning

Book Rating Prediction

An End-to-End Machine Learning Pipeline
on the Goodreads Dataset

Prepared by

Mayssem SAAD

Veronica CASTILLO

Hugo MAHIOU

Elias EL FERKH

Supervisor: Hanna Abi Akl

Academic Year: 2026

Contents

1	Methodology	2
1.1	Dataset, exploration and cleaning	2
1.1.1	Target analysis	2
1.1.2	Attributes analysis	3
1.1.3	Attributes correlation with average_rating	4
1.2	Feature engineering	4
1.3	Candidate models	5
1.4	Split, 5-fold Cross-Validation and evaluation protocol	6
1.5	Parameter tuning and final fit	6
2	Results	7
2.1	Model comparison and best performance	7
2.1.1	Five-fold cross-validation evaluation	7
2.1.2	Final evaluation	7
2.2	Feature impact analysis	8
2.3	Extended dataset training	8
2.3.1	Matched holdout performance	9
2.3.2	Leakage control and calibration	9
3	Deployment	10
3.1	Application Architecture	10
3.2	BookWise Interfaces	11
3.3	Prediction Interface	11
4	Conclusion	12
4.1	Limitations and Future Work	12

Introduction

In this group project, we built an end-to-end machine learning pipeline to predict a book's average rating from metadata and reader-engagement information. Online book communities such as Goodreads collect millions of ratings from readers. The average rating provides a simple measure of how a book is perceived, but it can be unstable when only a small number of readers have rated a book.

The initial Goodreads dataset contained 11,127 rows. An extended dataset from the same platform was then used to refit the selected model on 444,075 leakage-controlled training rows and measure the change in predictive accuracy. The prediction target, `average_rating`, is continuous on a 0–5 scale, making this a **supervised regression** task. The eight raw model inputs are *title*, *authors*, *language_code*, *num_pages*, *ratings_count*, *text_reviews_count*, *publication_date*, and *publisher*. Identifiers such as ISBN and bookID were not used as predictive features.

Performance is evaluated mainly using Mean Absolute Error (MAE). Root Mean Squared Error (RMSE) and R^2 are used as complementary metrics. The raw dataset requires preprocessing due to missing values, inconsistent formats, parsing errors, and duplicated editions. We follow an end-to-end workflow: data cleaning, feature engineering, regression model comparison, and deployment, with particular attention to avoiding data leakage.

The full machine learning pipeline was built on a Jupyter notebook¹. The best model from our analysis was then exported (with `cloudpickle`) and loaded into a web application to be tested through a user interface. The deployed interface is shown in a demo video². A GitHub repository³ was set up and includes the Jupyter notebook, the source code for the web application, and the exported models.

<https://bookwise-client.onrender.com>

The remainder of this report is organized as follows: the first chapter presents the methodology, the second chapter presents the experimental results, the third chapter describes the BookWise application, and finally the conclusion discusses limitations and possible improvements.

Generative AI tools, including large language models (LLMs), were used to assist with code generation, debugging, and text drafting. All outputs were reviewed and verified by the project group, which remains fully responsible for the final content.

¹https://github.com/hyougo/Goodreads_Ratings_ML/blob/main/notebooks/goodreads_ML_pipeline_final.ipynb

²https://github.com/hyougo/Goodreads_Ratings_ML/blob/main/docs/delivrables/ML%20webapp%20video.mp4

³https://github.com/hyougo/Goodreads_Ratings_ML

Chapter 1

Methodology

This chapter describes the full modelling pipeline. It covers data cleaning and exploration, feature engineering, candidate model selection, the train–test split with cross-validation protocol, and the hyperparameter tuning that produced the final model.



Figure 1.1: End-to-end workflow from repaired data to the deployed BookWise application.

1.1 Dataset, exploration and cleaning

The dataset contained 4 malformed rows, repaired manually before loading. Visual inspection showed further issues: whitespace inconsistencies, impossible publication dates, zero ratings, zero pages, and authors missing with ‘NOT A BOOK’ value. No true duplicates were found (all ISBNs unique).

A cleaning process standardized string values (whitespaces, lowercase normalization) before data analysis.

1.1.1 Target analysis

Main takeaway: Average ratings are concentrated: mean 3.93, median 3.96, middle 90% from 3.44 to 4.38. The dataset contains almost no very low rated books, so any model will struggle predicting extreme ratings.

Ratings based on few readers are much less stable. The standard deviation is 0.526 for books with 1–9 ratings and falls to 0.281 in the 50–99 band. We therefore require at least 50 ratings for any model to learn from, removing 1,883 rows but giving a more reliable target.

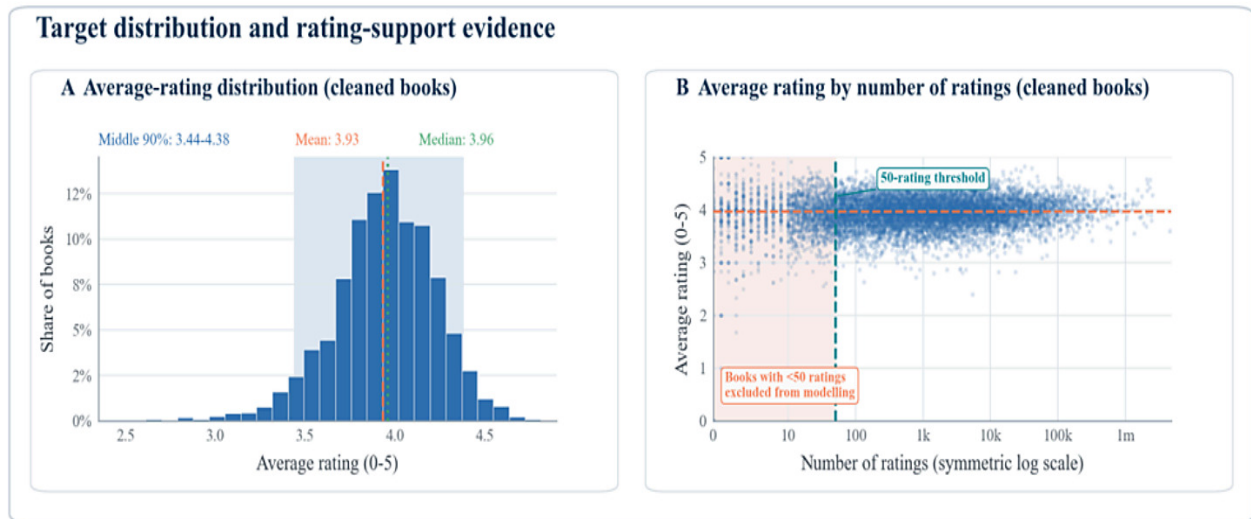


Figure 1.2: Target distribution and rating-support evidence. The shaded area contains books excluded from modelling, and the vertical line marks the 50-rating threshold.

1.1.2 Attributes analysis

Each attribute was inspected thoroughly in order to better understand the data and find weak spots, useful patterns, and feature ideas.

1. **ratings_count**: a few books with zero rating (and sometimes with a positive *average_rating*) were found and excluded from the dataset. After comparing rating stability across support bands, we excluded books with fewer than 50 ratings.
2. **text_reviews_count**: strongly right-skewed, a log scale was used to smooth that effect. Books with zero text reviews are kept (assuming they have more than 50 ratings). An additional *text_reviews_count* to *ratings_count* ratio was derived as a feature.
3. **publication_date**: ranges from 1900 to 2016, with very few books in the 1900–1950 range. For feature engineering, only the year is kept.
4. **num_pages**: several books with zero pages were found, most were identified as audiobooks through the publisher name or the title but some were not. This clue allowed us to define a pattern to identify audiobooks in the dataset. We decided to keep the zero-page entries with a *num_pages_missing* flag.
5. **language_code**: more than 90% of books in the dataset are English books (including US-GB-CCA variant). Several languages have very low representation. We decided to apply a threshold of 20 books for a language to be kept in the feature engineering and one-hot encoded, the rest is folded into an “other” category.
6. **authors**: this attribute seems to list not only authors, but all the contributors to the book (some books have more than 50 “authors”). In order to simplify, we decided to keep only the first listed author (*first_author*), which appears to be the main author of the book, as well as the number of listed authors. There are too many unique *first_author* (high cardinality) to realistically one-hot encode this feature, but since this attribute is likely the main driver for the *average_rating*, we decided to implement a safe target encoding during the modelling phase, i.e., mapping *first_author* to a cross-fitted mean *average_rating* across the learning dataset. We also computed the *average_rating* standard deviation and frequency of that author within the learning dataset as a feature.
7. **title**: the dataset contains duplicate titles, but each with a unique ISBN, they vary only by publication date, language, publisher or contributors (“authors”). Multiple patterns can be identified in the titles, such as audio books, books that are part of a series or books that are not single books

but a collection box. These patterns have been used to define new features (*is_audio*, *is_collection*, *is_series*). There are too many different titles to one-hot encode this attribute, but we can easily derive numerical features like number of characters or number of words. This seems counterintuitive as an *average_rating* signal but it can carry some information (related to genre, etc.).

8. **publisher**: the number of unique publishers again prohibits the one-hot encoding method, but this attribute is the main driver for audio-books identification. Similar to authors, we decided to apply a safe target & frequency encoding during modelling.

1.1.3 Attributes correlation with *average_rating*

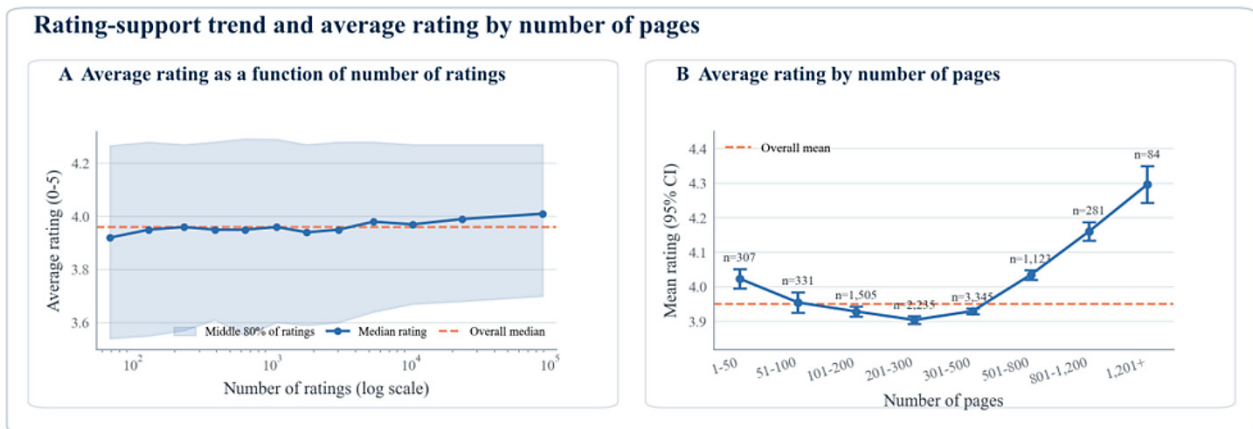


Figure 1.3: Rating-support trend and average rating by number of pages. The left panel shows the median and middle 80%; the right panel shows means, 95% confidence intervals, and band sample sizes.

None of the numerical features have a strong linear correlation with *average_ratings*. The right panel of Figure 1.3 shows a non-linear page-count relationship: mean ratings are similar through the central bands, then rise above 500 pages. The longest bands contain fewer books and include multi-volume collections, so this is an association rather than evidence that length causes higher ratings. This supported using tree-based models; we expect author identity to carry much of the useful signal.

1.2 Feature engineering

From eight raw fields, a deterministic feature builder converts them into 15 feature groups. This stage normalizes text, parses dates and numbers, and creates signals matching the data issues seen during exploration. The 15 groups become 24 processed columns: 12 numeric values, six author/publisher statistics, and six language indicators. Keeping the feature builder inside the exported pipeline lets the web-app send the same eight raw fields used during training. The complete lineage is shown in Figure 1.4.



Figure 1.4: Feature lineage from eight raw inputs to 15 engineered groups and 24 processed columns.

Leakage control: feature construction is stored in the pipeline, while imputation and all learned encoders are fitted only on the current training data or fold. Figure 1.5 shows this fold-fitted sequence.

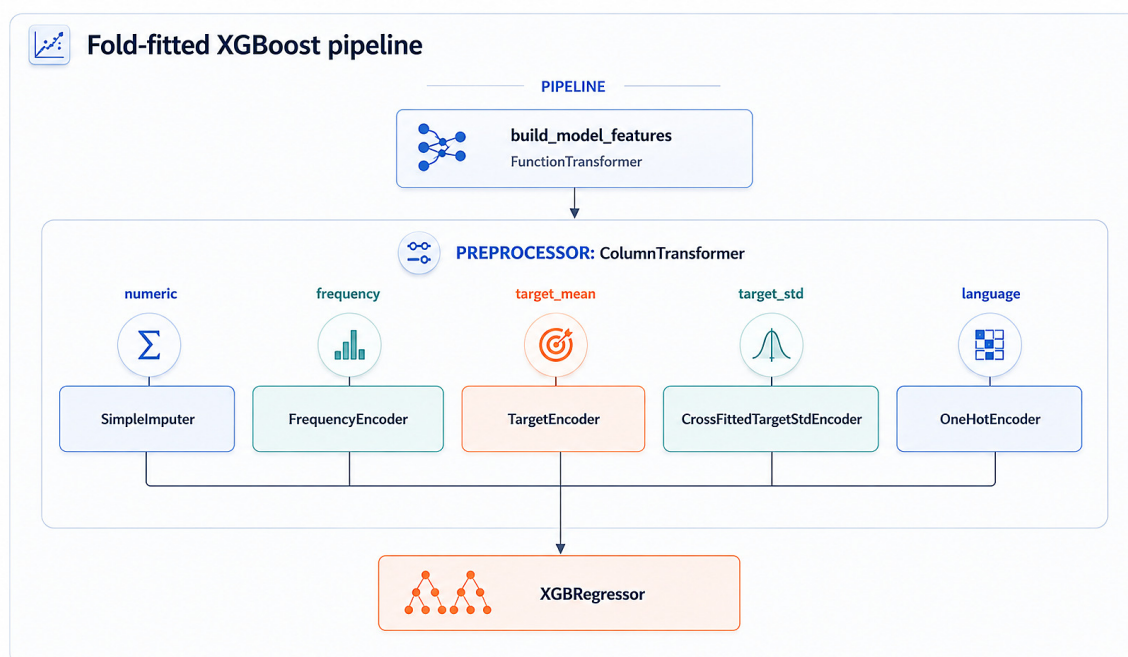


Figure 1.5: Fold-fitted XGBoost pipeline. Learned preprocessing remains inside each training fold.

1.3 Candidate models

Three regression algorithms were selected to represent complementary modelling approaches, from a simple linear model to gradient boosting methods. Their roles are summarized in Table 1.1.

Table 1.1: Candidate regression models and their role in the comparison.

Model	Role in the comparison
Lasso	Interpretable linear benchmark with regularization.
XGBoost	Non-linear boosted trees on the engineered and encoded columns.
CatBoost	Boosted trees using normalized categorical identities directly.
Mean baseline	Same mean prediction for every book.

Lasso is a regularized linear reference showing whether additive relationships suffice. **XGBoost** can learn non-linear relationships and interactions between numeric and encoded features. **CatBoost** offers a second boosted-tree approach with native handling of categories. A **mean-rating dummy regressor** provides the minimum baseline that a useful model should beat.

1.4 Split, 5-fold Cross-Validation and evaluation protocol

The 9,239 remaining books were split before any encoder, imputer, scaler, or estimator was fitted. We used a fixed **85%/15% train-test split** (7,853 training and 1,386 test books). Quantile bands (10) of the target stratified the split. The test set was held out for final evaluation, while all model selection and hyperparameter tuning were performed exclusively on the training set. Table 1.2 confirms the target distribution remained balanced across the two subsets.

Table 1.2: Balance of the training and holdout sets after target-decile stratification.

Split	Rows	Mean	Median	Std.
Training	7,853	3.951	3.96	0.268
Holdout	1,386	3.952	3.96	0.265

NB: *num_pages*, *ratings_count* and *text_reviews_count* also have close enough median values between train and test dataset.

Within the training data, a **shuffled five-fold splitter with random_state=42** was reused for every model. Each pipeline was fitted five times; four folds supplied training data and the remaining fold validation data, with the average score providing a robust estimate of generalization. Because preprocessing is part of the pipeline, encoders are refitted inside each fold and cannot learn from that fold's validation ratings. Using the same folds for all candidates makes their scores directly comparable.

We used Mean Absolute Error (MAE) as the main selection measure because it is easy to interpret: an MAE of 0.16 means an average absolute miss of about 0.16 on the 0–5 scale. Root Mean Squared Error (RMSE) gives more weight to large misses. R^2 compares the model with one that always predicts the mean rating: $R^2 = 0$ means no improvement, while a higher value means more variation is explained. To compare the 3 models on their best performances, we implemented a hyperparameter tuning process using 5-fold cross-validation and `random_state`.

1.5 Parameter tuning and final fit

After testing 24 hyperparameter combinations per model using five-fold cross-validation and MAE as the selection metric, XGBoost was selected as the best-performing model. We then refitted its complete pipeline on all 7,853 training rows and assessed its final performance on the remaining 1,386-book holdout set.

Chapter 2

Results

This chapter compares the candidate models, identifies the most influential features, and presents the extended-data retraining.

2.1 Model comparison and best performance

This section compares all candidate models and identifies the best-performing configuration based on cross-validation and holdout evaluation.

2.1.1 Five-fold cross-validation evaluation

XGBoost achieved the lowest five-fold MAE (0.1577), compared with 0.1659 for CatBoost, 0.1673 for Lasso, and 0.2099 for the mean baseline. The fold standard deviation was only 0.0031 (Figure 2.1).

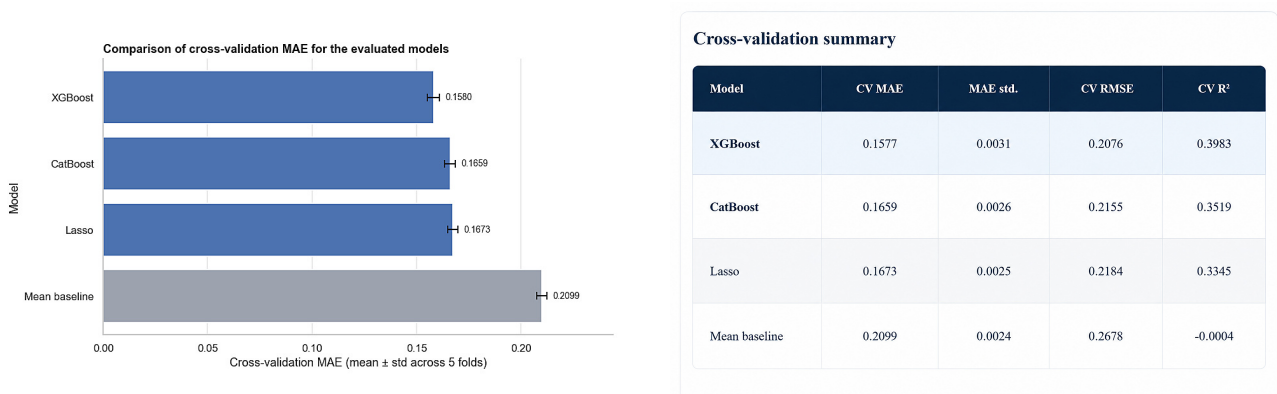


Figure 2.1: Five-fold cross-validation comparison. XGBoost achieved the lowest average validation error.

2.1.2 Final evaluation

The initial XGBoost model was evaluated on the unchanged original holdout set ($n = 1,386$). Its performance was an MAE of 0.1528, an RMSE of 0.2018, and an R^2 of 0.4205.

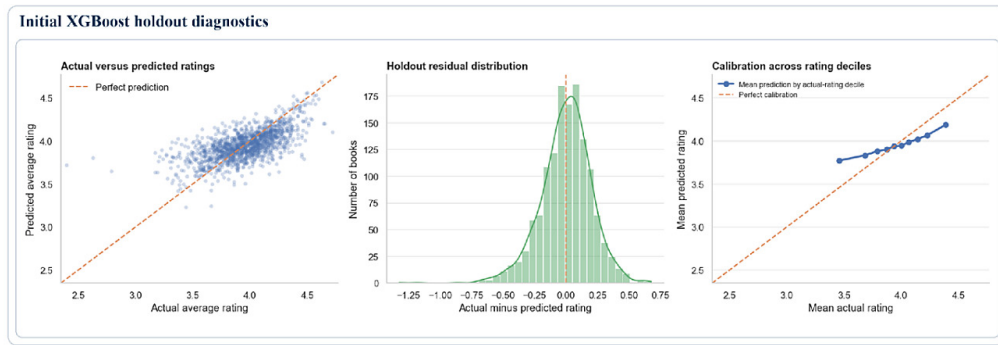


Figure 2.2: Initial XGBoost holdout diagnostics: predictions, residuals, and calibration by decile.

Residuals are centered close to zero, but the calibration panel reveals regression toward the mean: low ratings are predicted too high and high ratings too low.

2.2 Feature impact analysis

Gain-based importance scores were extracted from the final XGBoost pipeline and aggregated by original feature (Figure 2.3). `first_author` was the strongest feature, followed by language and publisher, while page information, title structure, and reader-engagement features contributed less.

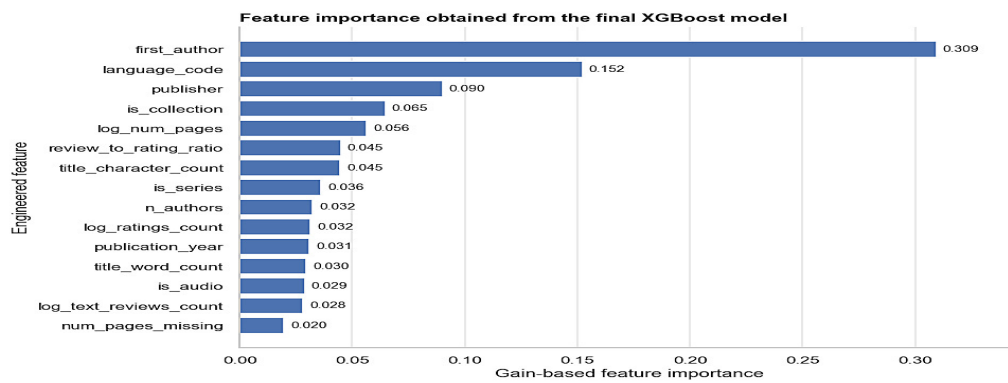


Figure 2.3: Gain-based importance of the engineered features in the final XGBoost model.

2.3 Extended dataset training

The extended dataset comes from the UCSD Goodreads release ¹. It contains 2,360,655 books, of which 443,951 remain after cleaning and filtering for at least 50 ratings. Figure 2.4 shows that this population reaches closer to both rating extremes, has lower typical reader support, and has 63.9% missing language codes.

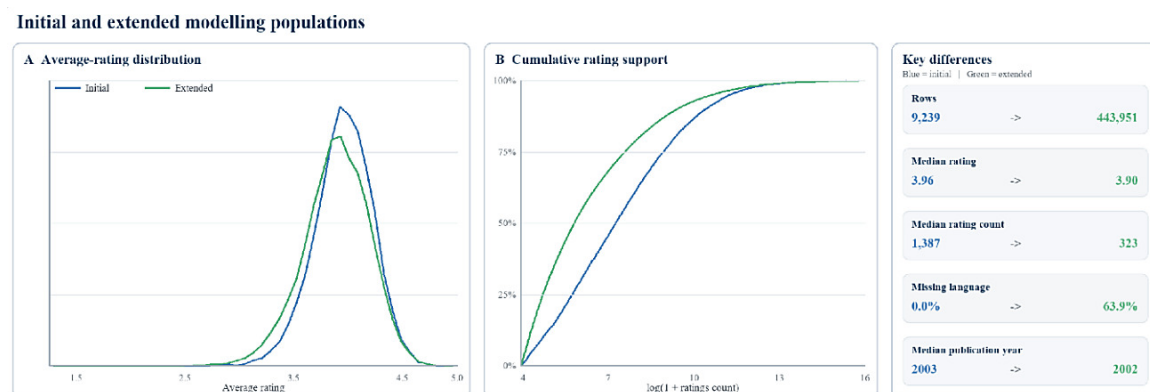


Figure 2.4: Comparison of the initial and extended modelling populations.

¹Wan et al., *Item Recommendation on Monotonic Behavior Chains*, RecSys 2018; Wan et al., *Fine-Grained Spoiler Detection from Large-Scale Review Corpora*, ACL 2019.

The original holdout was kept unchanged: no test ISBN enters the larger training set. After removing 1,260 overlapping rows and adding 1,384 original-training rows, the final training population contained 444,075 rows. The same XGBoost configuration was refitted without additional tuning.

2.3.1 Matched holdout performance

Table 2.1 summarizes the final XGBoost performance on the same original holdout. The extended model clearly improves all three evaluation metrics.

Table 2.1: Final XGBoost performance on the same original holdout ($n = 1,386$).

Training fit	Training rows	MAE	RMSE	R^2
Initial XGBoost	7,853	0.1528	0.2018	0.4205
Extended XGBoost (deployed)	444,075	0.1331	0.1748	0.5653
Change	+436,222	-12.9%	-13.4%	+0.1448

The broader training set substantially improved performance on the unchanged holdout. MAE decreased from 0.1528 to 0.1331, while RMSE decreased from 0.2018 to 0.1748. At the same time, R^2 increased from 0.4205 to 0.5653. These results show that the improvement was obtained without changing the evaluation set or introducing additional hyperparameter tuning.

2.3.2 Leakage control and calibration

The leakage-controlled training-set construction and the corresponding holdout improvement are shown in Figure 2.5. The calibration and prediction spread before and after extended-data training are shown in Figure 2.6.

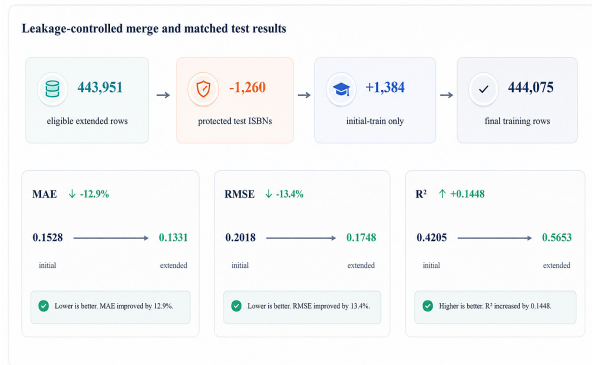


Figure 2.5: Leakage-controlled extended training-set construction and matched holdout improvement.

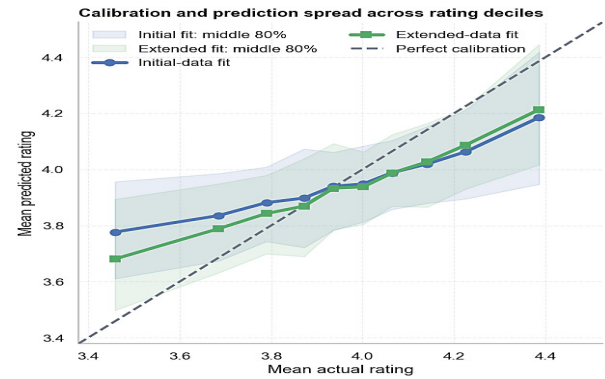


Figure 2.6: Calibration and prediction spread before and after extended-data training.

The broader fit moved the calibration closer to the ideal line, especially for the lowest rating groups, although both models still pull extreme ratings toward the centre (Figure 2.6). Overall, the extended-data model provides a clear improvement over the initial configuration while preserving the original holdout as a strictly independent evaluation set. The reduction in both MAE and RMSE, together with the increase in R^2 , supports the use of the extended-data XGBoost model as the final deployed configuration.

Chapter 3

Deployment

The XGBoost model was integrated into **BookWise**, a web application. The trained pipeline contains feature engineering, encoders, and fitted model. It achieves a holdout MAE of 0.133 and R^2 of 0.57. Predictions are clipped to $[0, 5]$.

3.1 Application Architecture

BookWise is a three-tier web application:

Table 3.1: BookWise application stack.

Tier	Role and technology
ML service	Python <i>FastAPI</i> microservice exposing prediction endpoints.
API server	<i>Node + TypeScript</i> (Express, Prisma, SQLite) for authentication, catalogue, search and recommendations.
Web client	<i>React + TypeScript</i> (Vite, Tailwind) for the user interface.

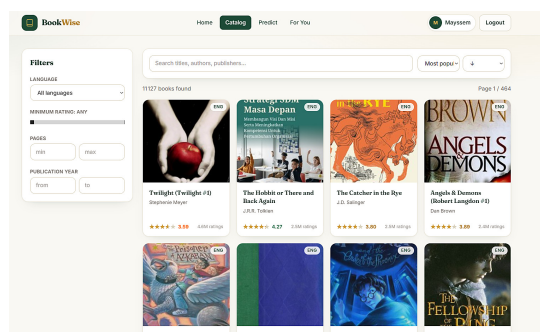
The ML service loads the serialized pipeline at startup and exposes a `/predict` endpoint that accepts JSON input with the eight raw book fields (title, authors, publisher, language, number of pages, publication date, ratings count, and text reviews count). The feature builder inside the pipeline transforms these inputs into the 24 processed columns before passing them to the XGBoost regressor. This design ensures that the web application does not need to replicate any preprocessing logic.

The API server handles user management, book storage, and search. It communicates with the ML service through internal HTTP calls. The web client provides the user-facing interface and is served as a single-page application.

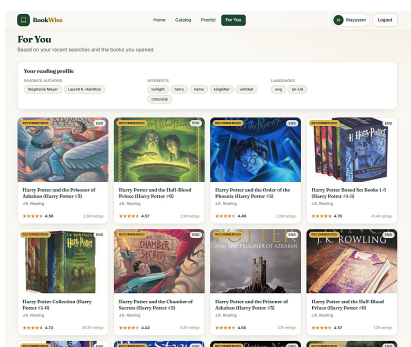
All three tiers are hosted on **Render** as separate services. The live application is accessible at <https://bookwise-client.onrender.com/predict>. To ensure continuous availability, an **UptimeRobot** monitor pings each service every 5 minutes and sends alerts if any tier becomes unreachable (<https://dashboard.uptimerobot.com/monitors/803721598>).

The image shows the BookWise homepage. At the top is a navigation bar with the BookWise logo and links for Home, Catalog, Predict, For You, MySystem, and Logout. The main banner features a background of bookshelves. The headline reads: "Discover your next read and *predict its rating* before the crowd." Below this, a paragraph states: "Explore a curated catalog of 11,000+ books, search with advanced filters, and get recommendations tailored to your reading — plus a smart estimate of any book's rating before you commit." At the bottom of the banner are two input fields: "Search a title, author or publisher..." and "Explore catalog...". Below these are two buttons: "Try the rating predictor" and "Personalized picks".

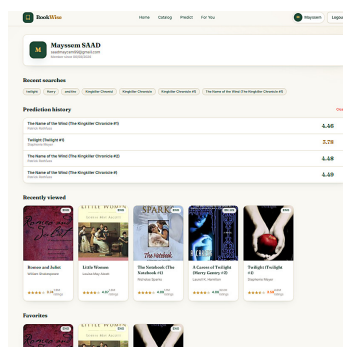
(a) Home page



(b) Book catalogue



(c) Personalised recommendations



(d) User profile

Figure 3.1: Main BookWise interfaces: home, catalogue, recommendations, and profile.

Users enter a book’s attributes and the system returns an estimated rating (0–5), displayed as a numerical score and star gauge. The prediction uses the same preprocessing pipeline as training.

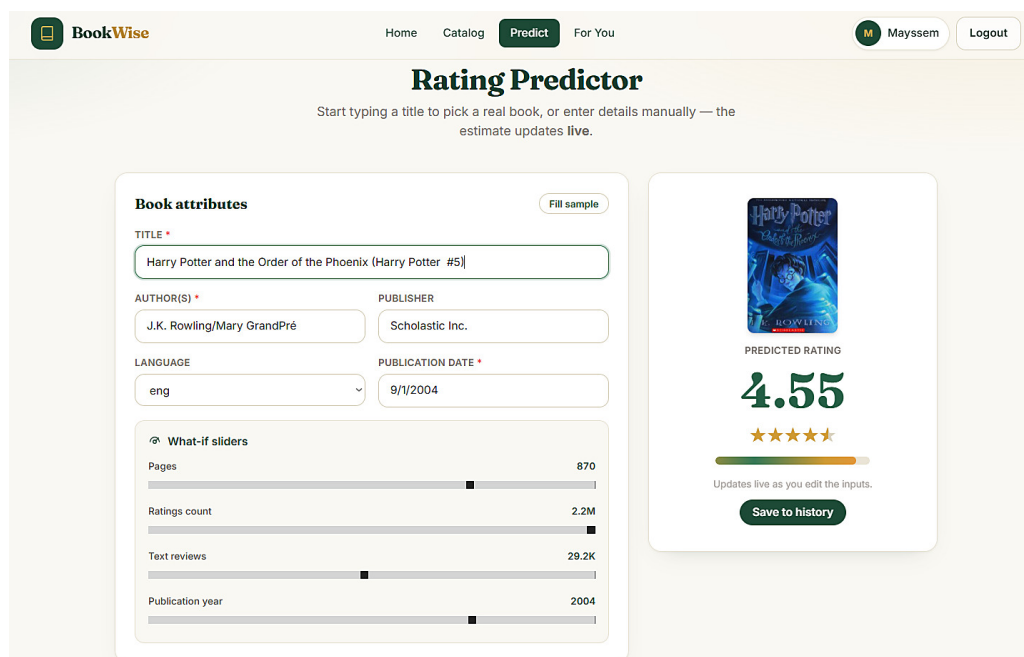


Figure 3.2: BookWise rating prediction interface.

Conclusion

This project predicted the average rating of a book using bibliographic information and reader engagement. The initial model achieved an MAE of about **0.15 rating points**, roughly a 26% improvement over predicting the dataset mean.

After retraining on a larger leakage-controlled Goodreads dataset, the MAE decreased to about **0.13**, with $R^2 \approx 0.57$. This extended-data model was used for deployment.

The most useful information came from historical rating patterns of authors and publishers, calculated using only training data to avoid leakage. Feature engineering played a more important role than algorithm choice: XGBoost and CatBoost produced close results, while the engineered features provided most of the predictive signal. The final model was packaged into a reproducible pipeline and integrated into the BookWise web application.

4.1 Limitations and Future Work

The main limitation comes from the data. Most books have ratings between 3.6 and 4.3, while extreme ratings are rare. The model has few examples of these cases, so its predictions tend toward the average and are less accurate at the extremes.

Additional records from online sources made the dataset much larger, but the rating distribution remained similar. XGBoost and CatBoost achieved very similar error rates despite different learning approaches, suggesting that data is a bigger limitation than algorithm choice. Lasso performed less well because it cannot capture non-linear relationships.

One important limitation concerns the extended dataset split: test books were excluded by ISBN, but the same work can have several ISBNs. Another edition of a test book may still appear in training. The improvement from 0.15 to 0.13 MAE may therefore be somewhat optimistic. A work-level split based on title and first author would give a more reliable estimate.

Future improvements could include:

1. work-level train/test split based on title and first author;
2. richer title/description features (embeddings, genre, subject);
3. calibration to improve performance at rating extremes;
4. collecting more books with extreme ratings;
5. additional hyperparameter tuning and model combination;
6. batch prediction through the BookWise interface via CSV upload.